Campus Event Management & Feedback Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Name	Student Code
26102410044	26102000166	SIPRA KUNDU	BWU/MCA/26/049

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

IN

COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

NOVEMBER & 2026

ABSTRACT

The Campus Event Management & Feedback Analyzer is a centralized web platform that replaces paper-based event registration, scattered announcements and unmeasured event outcomes with one system for administrators and students. Universities run many workshops, hackathons, cultural fests and sports events, but rarely measure how well each one performed.

The system is built on a Clean Layered Architecture that separates the presentation, service, domain and data-access layers. A 10-step smart registration pipeline checks the student, the event, its status, the registration deadline, the remaining seats and duplicate bookings before it issues a ticket. After an event, students who attended can submit a 1 to 5 star review with a written comment.

Pandas and NumPy power the analytics. They compute the mean, median, standard deviation, variance and interquartile range of student ratings. They also compute a multi-factor Event Performance Score from 0 to 100, which weights seat utilization (40%), average rating (40%) and feedback engagement (20%). Eight Matplotlib charts and a real-time Plotly Feedback Analyzer, with movable charts and live filters, turn the data into visual insight.

The application is written in Python 3.10+ with a Streamlit interface and stores data in CSV files behind a repository pattern, so it can move to PostgreSQL or MySQL without changing the services or the UI. Twenty automated unit tests pass with no failures. The platform gives administrators a clear, data-backed view of campus engagement and gives students a simple way to discover, book and review events.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	Abstract	i
	Table of Contents	ii
1	Introduction	1
2	Project Overview	2-3
3	System Design & Architecture	4-5
4	Implementation	6-8
5	UI/UX Design	9-11
6	Security, Privacy & Testing	12
7	Results & Discussion	13
8	Conclusion & Future Scope	14
9	References	15

1. INTRODUCTION

1.1 General

A university runs a steady stream of events: technical workshops, hackathons, cultural fests, sports championships and startup pitch days. Many institutions still handle them with paper forms, notice boards and messaging groups. Organizers cannot easily see how many seats are left, students miss deadlines, and nobody measures whether an event was worth running.

Two gaps stand out. First, registration is uncontrolled: duplicate bookings, over-filled halls and late sign-ups are common. Second, outcomes are not quantified: student opinions are rarely collected, and when they are, they are not turned into numbers an administrator can compare across events.

The Campus Event Management & Feedback Analyzer closes both gaps. It manages the whole event lifecycle from scheduling to registration, attendance and review, and it converts participation and feedback data into statistics, charts and a single performance score per event.

1.2 Objectives

The primary objectives of this project are:

- To replace paper-based registration with a single online system for administrators and students.
- To enforce registration rules automatically: capacity, deadline, event status and duplicate prevention.
- To collect structured post-event feedback from students who actually attended.
- To compute descriptive statistics (mean, median, standard deviation, variance, IQR) on student ratings.
- To score every event from 0 to 100 and group events into performance tiers.
- To present results through bar charts, histograms, donut charts and real-time interactive Plotly charts.
- To keep the architecture layered and portable so the storage backend can change without touching the business logic.

2. PROJECT OVERVIEW

2.1 System Objective

The system is designed to work as three roles in one tool:

- **Event Coordinator:** schedules events, tracks capacity and moves events through their lifecycle.
- **Gatekeeper:** validates every registration against strict rules before issuing a ticket.
- **Analyst:** turns registrations and reviews into statistics, charts and performance scores.

2.2 Stakeholders

Stakeholder	Role in the system
Administrator	Schedules events, manages students, marks attendance, reviews analytics and exports reports
Student	Browses events, books seats, manages tickets and reviews attended events
Department of Computer Applications	Academic guide and mentor for the project

2.3 Scope of the System

The system covers event scheduling, student enrollment, smart registration, attendance, post-event feedback, statistical analysis, visualization and report export. Events move through the states Open, Full, Completed and Closed. Out of scope for this version are QR-code check-in, certificates and a relational database; they appear in the future scope.

2.4 User Roles and Capabilities

Administrator

- Command Center dashboard with KPI cards, a top-performing event spotlight and activity feeds.
- Event management: create, edit, cancel or complete events and monitor capacity.
- Student directory: search by department, course or semester, and enroll students with duplicate protection.
- Registration and attendance roster: check ticket status and mark attendance on event day.
- Analytics: visualizations, statistical analysis, the Feedback Analyzer and Event Performance Scoring.
- Reports and exports in `.txt` and `.csv` formats.

Student

- Personal dashboard with upcoming bookings, attended events, submitted reviews and pending reviews.
- Event discovery with search filters, capacity progress meters and countdowns.
- One-click booking with instant validation feedback.
- Ticket management, including cancellation to release a seat.
- Feedback with a 1 to 5 star rating and a comment, only for completed events the student attended.

3. SYSTEM DESIGN & ARCHITECTURE

3.1 Clean Layered Architecture

The system has five layers. Each layer calls only the layer directly below it, so the user interface holds no business logic and the storage can be replaced without touching the services.

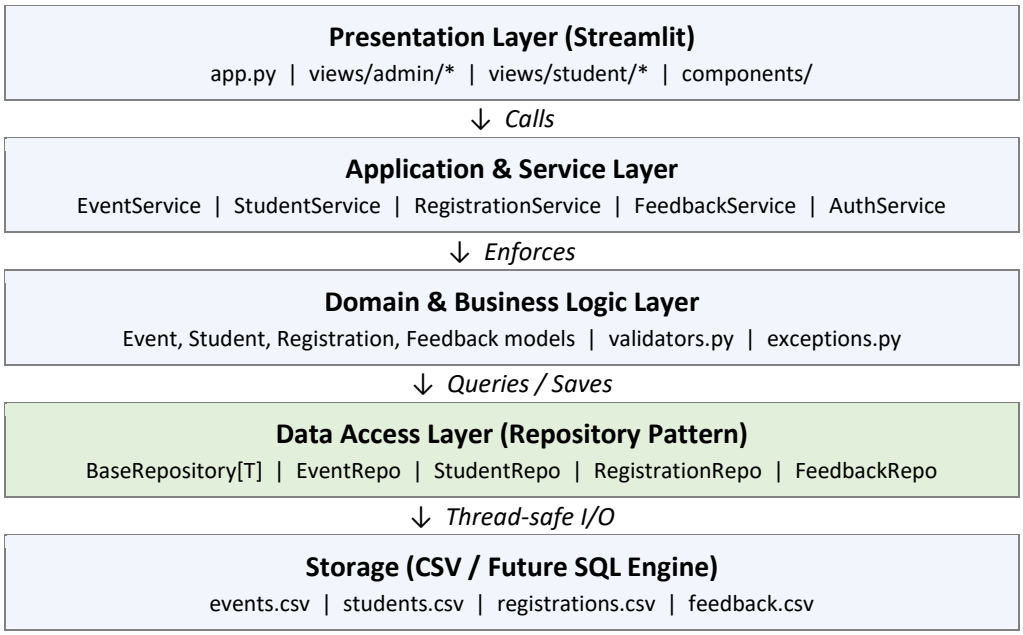


Figure 3.1: Layered architecture of the system

Layer	Components	Responsibility
Presentation	`app.py`, `views/admin/*`, `views/student/*`, `components/`	Streamlit pages, role-based routing, KPI cards and forms
Application & Service	`EventService`, `StudentService`, `RegistrationService`, `FeedbackService`, `AuthService`	Use cases such as booking, reviewing and logging in
Domain & Business Logic	`Event`, `Student`, `Registration`, `Feedback` models, `validators.py`, `exceptions.py`	Entities, validation rules and custom domain exceptions
Data Access	`BaseRepository[T]` and four repositories	Thread-safe reads and writes behind the Repository Pattern
Storage	`events.csv`, `students.csv`, `registrations.csv`, `feedback.csv`	Persistent data, replaceable by PostgreSQL or MySQL

3.2 Project Directory Structure

```
Campus Event Management & Feedback Analyzer/
|-- app.py              # Streamlit application and role-based router
|-- config.py           # Constants, score weights and paths
|-- requirements.txt    # Streamlit, Pandas, NumPy, Matplotlib, Plotly
|-- models/             # event, student, registration, feedback dataclasses
|-- repositories/       # BaseRepository[T] and four CSV repositories
|-- services/           # event, student, registration, feedback, auth services
|-- analytics/          # event, participation and feedback analytics
|-- visualization/      # charts.py with 8 chart routines
|-- components/         # ui_helpers.py, auth_ui.py
|-- views/admin/        # dashboard, events, students, registrations,
|                       #   analytics, reports
|-- views/student/      # dashboard, browse, my events, feedback
|-- utils/              # constants, exceptions, helpers, validators, seed_data
|-- data/               # events, students, registrations, feedback CSV files
|-- reports/generated_reports/
`-- tests/              # test_events, test_students,
                        #   test_registrations, test_feedback
```

3.3 Smart Registration Pipeline

Every booking passes ten checks in order. The first failure stops the process and raises a specific exception.

1. Validate the student ID.
2. Validate the event ID.
3. Check the event status. Cancelled, Draft or Closed events raise `RegistrationClosedError`.
4. Check the deadline. A passed deadline raises `RegistrationClosedError` and updates the event status.
5. Check capacity. A full event raises `EventFullError` and updates the status.
6. Check for a duplicate booking. An existing booking raises `DuplicateRegistrationError`.
7. Generate a unique registration ID.
8. Persist the registration record.
9. Re-check capacity and set the event to Full if the limit is now reached.
10. Return the confirmation receipt and display the ticket.

3.4 Database Portability

Because every service talks to `BaseRepository[T]`, the CSV backend can be replaced by an SQLAlchemy repository for PostgreSQL or MySQL. No service, validator or UI page needs to change.

4. IMPLEMENTATION

4.1 Technology Stack

Area	Technology
Language	Python 3.10 to 3.13
Web interface	Streamlit
Data processing	Pandas, NumPy
Static charts	Matplotlib
Interactive charts	Plotly
Storage	CSV files behind a thread-safe repository layer
Testing	Python `unittest`

4.2 Domain Models and Services

The four domain models (`Event`, `Student`, `Registration`, `Feedback`) are plain dataclasses. Student passwords are stored as SHA-256 hashes. Five services hold the business rules. `EventService` handles scheduling, capacity synchronization and the event state machine (Open, Full, Completed, Closed). `StudentService` manages the student lifecycle and validation. `RegistrationService` runs the 10-step pipeline. `FeedbackService` allows a review only for a completed event that the student attended, once per event, with a rating from 1 to 5. `AuthService` handles sessions and hashing.

4.3 Analytical Engines

The Pandas engine joins `registrations.csv`, `students.csv` and `events.csv`. It computes department-wise enrollment percentages, category distributions and venue utilization. The NumPy engine works directly on rating arrays and returns these statistics:

$$\mu = (1/N)\,\Sigma\, x_i \qquad \sigma = \sqrt{[\,(1/N)\,\Sigma\,(x_i - \mu)^2\,]} \qquad IQR = Q_3 - Q_1$$

- **Arithmetic mean:** the overall satisfaction level.
- **Median:** the middle rating, resistant to outliers.
- **Standard deviation and variance:** how widely opinions are spread.
- **Quantiles and interquartile range (IQR = Q3 - Q1):** the spread of the middle 50% of ratings.

4.4 Event Performance Score

The Event Performance Score (EPS) combines three normalized measures into one number from 0 to 100.

$$EPS = 0.40 \times S_{capacity} + 0.40 \times S_{rating} + 0.20 \times S_{engagement}$$

$$S_{capacity} = min(1, Registered / Capacity) \times 100$$

$$S_{rating} = (Average\ Rating / 5) \times 100$$

$$S_{engagement} = min(1, Reviews / Registered) \times 100$$

Tier	EPS range	Meaning
Top Performer	70.0 and above	High turnout, strong satisfaction, good feedback engagement
Average Performer	40.0 to below 70.0	Moderate participation or average ratings
Needs Improvement	Below 40.0	Low attendance or low satisfaction; needs review

4.5 Visualization Suite

`visualization/charts.py` provides eight chart routines, shown in a gallery and a single-chart inspector. Every chart can be downloaded as a high-resolution PNG.

Chart	Shows
Event Participation (bar)	Top events ranked by confirmed registrations
Rating Frequency (histogram)	Distribution of 1 to 5 star ratings with percentages
Popularity vs Capacity (grouped bar)	Registered seats beside maximum capacity
Average Event Ratings (benchmark bar)	Event ratings against a 4.0 star target line
Department Engagement (donut and bar)	Share of registrations by department
Sentiment Proportion (donut and bar)	Positive, neutral and critical share of reviews
Category Analysis (categorical bar)	Registrations and average turnout by event category
Performance Score Matrix	EPS of each event against tier thresholds

4.6 Real-Time Feedback Analyzer

The Feedback Analyzer uses Plotly charts that the user can pan by dragging, zoom by scrolling and inspect by hovering. It has three charts: a rating frequency bar chart, a sentiment donut (Positive 4 to 5 stars, Neutral 3 stars, Critical 1 to 2 stars) and a horizontal event benchmark bar. Filters for event, star rating and sentiment recalculate every chart and the review table at once. A reviews ledger lists the student name, ID, event, rating, sentiment badge and timestamp, and the filtered data can be exported to CSV.

4.7 Installation and Execution

The application needs Python 3.10 or newer. From the project folder:

```
pip install -r requirements.txt
streamlit run app.py
```

The application opens in the browser at `http://localhost:8501`. On first start, `seed_data.py` fills the data folder with sample events, students, registrations and reviews. Reports are written to `reports/generated_reports/` as `.txt` and `.csv` files.

5. UI/UX DESIGN

The interface uses Streamlit with custom CSS: a dark theme, a blue-to-teal hero banner on every page, white KPI cards, colored status badges and a sidebar for role-based navigation.

5.1 Login Screen

The landing page shows the application name, the tagline "Manage Events. Connect Students. Analyze Participation." and the university badge. Two tabs, Administrator Login and Student Login, open separate credential forms.

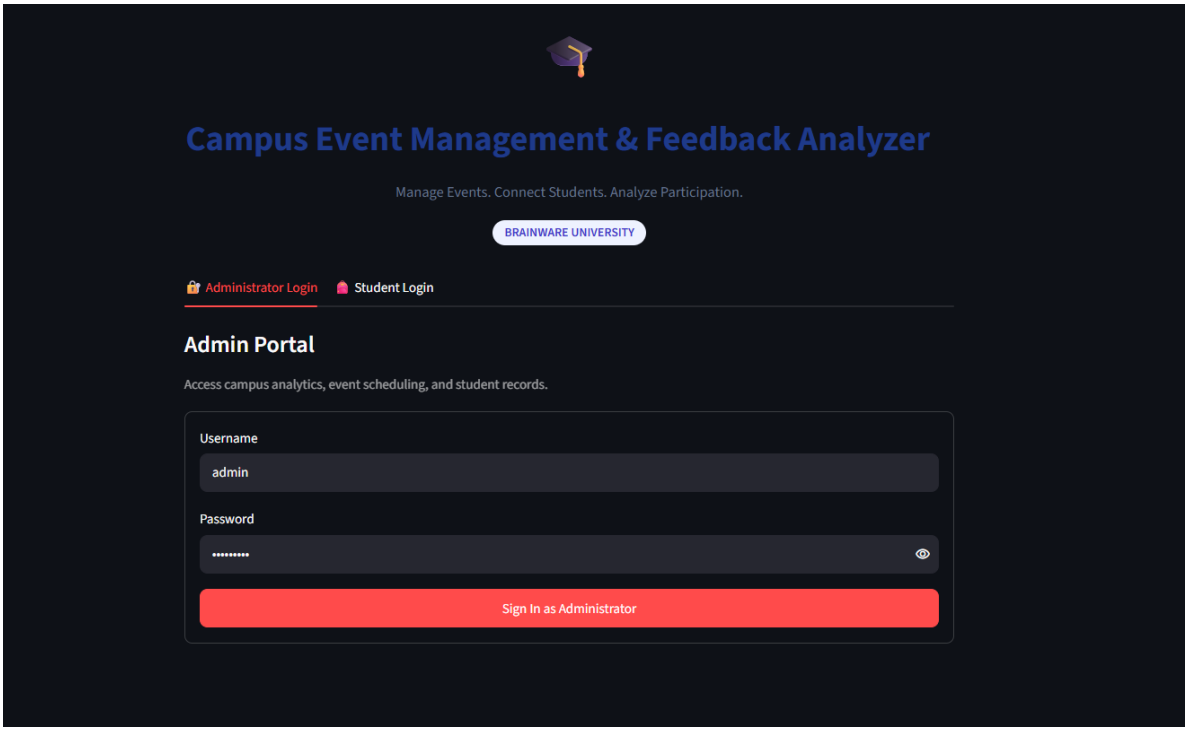


Figure 5.1: Administrator login screen

5.2 Administrator Command Center

The dashboard opens with four KPI cards: Total Events (15), Registered Students (41), Total Registrations (142) and Average Satisfaction (4.21 / 5.0). A highlighted card names the top-performing event, Full-Stack Web Development Masterclass, with a performance score of 74.0 out of 100. Below it are the upcoming events list, a table of recent registrations and the latest student reviews. The sidebar shows the signed-in user and six navigation entries:

Command Center, Event Management, Student Directory, Registrations & Attendance, Analytics & Visualizations, and Reports & Exports.

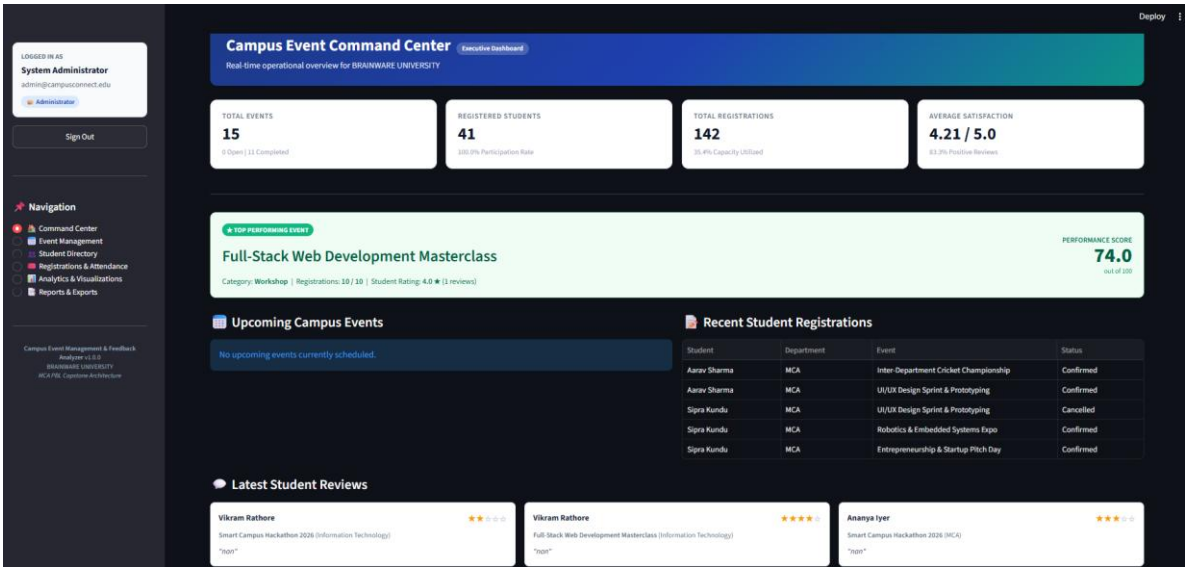


Figure 5.2: Administrator Command Center

5.3 Student Portal

The student dashboard greets the student by name and shows four cards: Upcoming Bookings, Attended Events, Reviews Submitted and Pending Reviews. A banner reminds the student when a completed event is waiting for feedback. The page lists upcoming registered events with category, date, venue and booking time, and shows recent reviews with star ratings. The sidebar offers My Dashboard, Browse Events, My Event Passes and Submit Feedback.

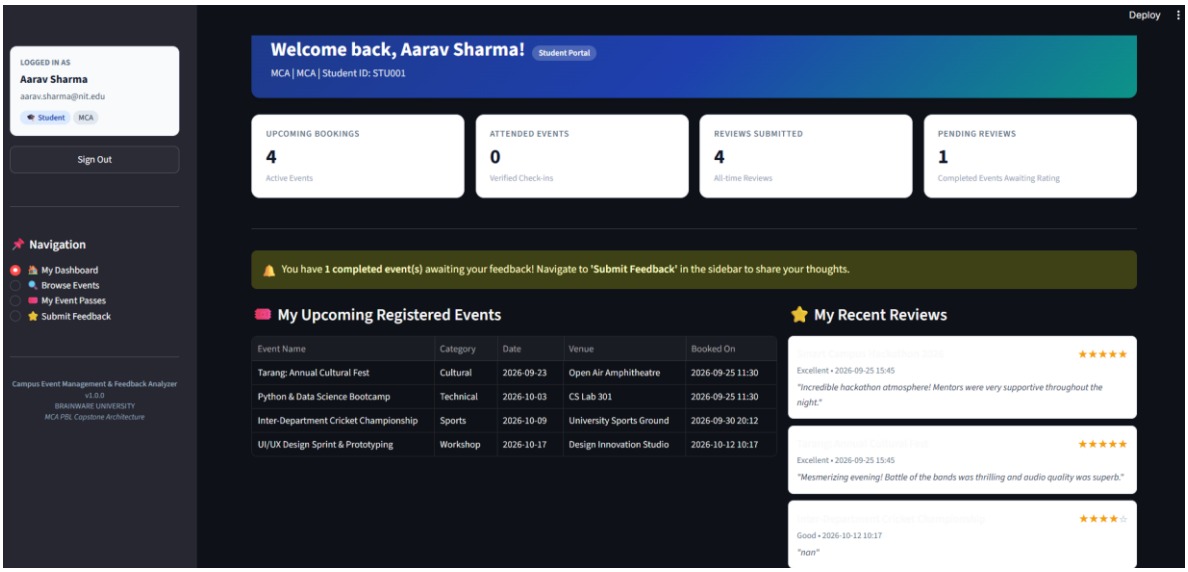


Figure 5.3: Student portal dashboard

5.4 Analytics and Feedback Analyzer

The Analytics Hub has five tabs: High-Level Metrics, Visualizations, Statistical Analysis, Feedback Analyzer and Event Performance Scoring. The Feedback Analyzer shows total reviews (42), overall rating (4.21), positive sentiment (83.3%) and critical or low ratings (2.4%). Three dropdown filters for event, rating and sentiment sit above the real-time movable Plotly charts: the rating frequency bar chart and the sentiment donut.

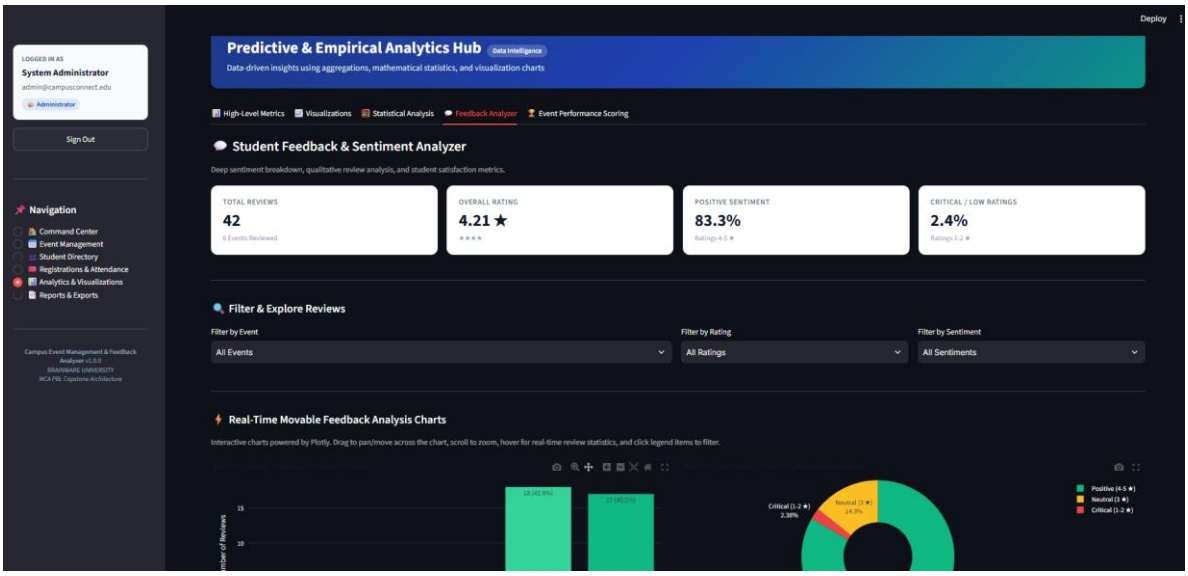


Figure 5.4: Feedback Analyzer with real-time charts

6. SECURITY, PRIVACY & TESTING

6.1 Security and Privacy

- **Role-based access:** the router in `app.py` shows administrator pages only to the administrator and student pages only to signed-in students.
- **Password hashing:** student passwords are stored as SHA-256 hashes, never as plain text.
- **Input validation:** `utils/validators.py` checks IDs, emails, dates, capacities and ratings before data reaches storage.
- **Business-rule enforcement:** duplicate bookings, over-capacity bookings, late registrations and reviews for events a student did not attend are rejected with custom exceptions.
- **Thread-safe storage:** the base repository serializes reads and writes to the CSV files.
- **Local data:** all data stays in the project data folder; nothing is sent to external servers.

6.2 Automated Testing

The unit test suite checks the business logic and the custom exceptions without altering production data. It runs with `python -m unittest discover -s tests -v`.

Test file	What it verifies
`test_events.py`	Valid event creation, duplicate event prevention, date format validation, capacity limits
`test_students.py`	Valid profile creation, unique student ID, unique email, syntax checks
`test_registrations.py`	Successful booking, duplicate rejection, full-capacity blocking, non-existent entities, seat restoration on cancellation
`test_feedback.py`	Valid submission, 1 to 5 rating boundaries, duplicate review rejection, completed-event requirement, unregistered-student blocking

Result: 20 tests ran and 20 passed, with 0 failures and 0 errors.

7. RESULTS & DISCUSSION

The system ran end to end on the seeded demonstration data and produced the figures below on the administrator dashboard and the Feedback Analyzer.

Measure	Observed value
Total events	15 (11 completed)
Registered students	41
Total registrations	142
Capacity utilized	35.4%
Average satisfaction	4.21 / 5.0
Total reviews	42 across 6 events
Positive reviews (4 to 5 stars)	83.3%
Neutral reviews (3 stars)	14.3%
Critical reviews (1 to 2 stars)	2.4%

7.1 Event Performance Score in Practice

The top-performing event, Full-Stack Web Development Masterclass, scored 74.0 out of 100, which places it in the Top Performer tier. The score can be checked by hand: it filled 10 of 10 seats (capacity score 100), has an average rating of 4.0 (rating score 80) and received 1 review for 10 registrations (engagement score 10). Then $0.40 \times 100 + 0.40 \times 80 + 0.20 \times 10 = 40 + 32 + 2 = 74.0$.

The example also shows what the engagement term does. An event with full seats and a good rating still loses points when few attendees leave a review, which pushes organizers to collect feedback.

7.2 Discussion

- **Registration control:** duplicate, late and over-capacity bookings are rejected by the pipeline, and the tests confirm it.
- **Insight:** the dashboard turns raw registrations and reviews into KPIs, charts and one comparable score per event.
- **Sentiment view:** with 83.3% positive and 2.4% critical reviews, satisfaction across the campus is high, and the filters let an administrator find the few low-rated events quickly.

7.3 Limitations

CSV storage suits a demonstration, not a large concurrent deployment. Sentiment is derived from star ratings only, not from the text of comments. The engagement score depends on how many students choose to review.

8. CONCLUSION & FUTURE SCOPE

8.1 Conclusion

The Campus Event Management & Feedback Analyzer replaces manual event handling with a single, rule-enforcing platform. It guarantees valid registrations through a 10-step pipeline, collects verified feedback and measures every event with statistics, eight chart types, a real-time Feedback Analyzer and a 0 to 100 performance score. The layered design keeps business logic out of the interface, and the passing test suite shows that the core rules hold.

8.2 Learning Outcomes

Building the platform gave practical experience across the full software development cycle:

- **Software architecture:** separating presentation, service, domain and data-access layers, and applying the Repository Pattern so storage can be replaced.
- **Data analysis:** using Pandas for multi-table joins and aggregations, and NumPy for mean, median, standard deviation, variance and IQR.
- **Data visualization:** drawing eight Matplotlib chart types and interactive Plotly charts that update with filters.
- **Rule-based validation:** turning real-world rules (capacity, deadlines, duplicates, attendance) into a registration pipeline with custom exceptions.
- **Testing:** writing 20 unit tests that check the business logic without touching production data.
- **User-centered design:** building separate administrator and student portals with role-based navigation and clear feedback messages.

8.3 Future Scope

The layered design allows the platform to grow without rewriting its core. The planned extensions are:

- **Relational database migration:** add an `SQLAlchemyRepository` implementing `BaseRepository[T]` for PostgreSQL or MySQL. Services, validators and pages stay unchanged, and concurrent use by many students becomes safe.
- **QR-code attendance:** issue a scannable QR code with every ticket so organizers can mark attendance from a phone instead of a manual roster.
- **Automated e-certificates:** generate personalized PDF participation certificates for every student marked as Attended.
- **AI-powered aspect sentiment analysis:** classify written comments by speaker quality, venue and schedule pace, so the written feedback adds to the star ratings.

9.REFERENCES

[1] Streamlit Documentation. https://docs.streamlit.io

[2] Pandas Documentation. https://pandas.pydata.org/docs

[3] NumPy Documentation. https://numpy.org/doc

[4] Matplotlib Documentation. https://matplotlib.org/stable

[5] Plotly Python Documentation. https://plotly.com/python

[6] Python unittest Documentation. https://docs.python.org/3/library/unittest.html

[7] Fowler, M. Patterns of Enterprise Application Architecture (Repository Pattern and layered architecture).

[8] Research literature on learning analytics and student engagement in higher education.

[9] Research literature on Likert-scale feedback analysis and composite indicators.